

Problema del productor-consumidor
sin sincronización

Nicolás Corbal Capas

Hugo Veiga Couselo

Introducción

La concurrencia es una característica fundamental en los sistemas operativos modernos y un factor clave para mejorar el rendimiento. Sin embargo, el acceso concurrente a recursos compartidos obliga a introducir mecanismos que garanticen la consistencia de los datos y eviten conflictos entre hilos. Cuando no se dispone de dichos mecanismos, las soluciones recurren a comprobaciones repetidas sobre variables compartidas, lo que se conoce como espera activa o busy waiting, una estrategia sencilla de implementar pero ineficiente y propensa a condiciones de carrera.

Uno de los problemas clásicos en este ámbito es el del productor-consumidor. En este escenario, un hilo productor genera datos que se almacenan en un buffer compartido, mientras que un hilo consumidor los extrae y procesa. Si no se utilizan primitivas de sincronización, los hilos deben coordinarse únicamente consultando el estado del buffer, lo que no garantiza la exclusión mutua en la región crítica y desperdicia ciclos de CPU mientras se espera una condición.

En el presente informe se documenta la implementación de una primera aproximación al problema productor-consumidor en lenguaje C, sin emplear mutexes ni variables de condición, de manera que la sincronización depende exclusivamente de la consulta de la variable cantidad del buffer. Se analiza el funcionamiento de la solución, las decisiones de diseño adoptadas y las limitaciones observadas, que servirán como punto de partida para la versión con mutexes desarrollada en el siguiente ejercicio.

Forma de ejecución

Primeramente, para compilar el código un ejemplo de comando de compilación usando el compilador gcc es:

```
gcc -Wall -I ./ sin_mutex.c buffer.c -o ejecutable -lpthread
```

Siendo el ejecutable ejecutado simplemente con:

```
./ejecutable
```

Funcionamiento del código

Las responsabilidades del código se reparten entre 3 archivos. El archivo de cabecera buffer.h, el archivo buffer.c con las funciones de producción y consumo y sin_mutex.c con la lógica funcional de la solución.

El archivo de cabecera `buffer.h` define las constantes de tamaño de buffer, el máximo de iteraciones y divide las iteraciones en 3 intervalos para introducir distintas velocidades. Además, contiene la estructura principal de datos con información como las posiciones de la cola, la cantidad de elementos presentes y los punteros a los hilos productor y consumidor, evitando por ahora el uso de mutexes o variables de condición.

El archivo `buffer.c` incluye las funciones de inserción y extracción del item, así como la función `produce_item()` encargada de leer números del archivo `numeros.txt`. Las operaciones de `insert_item()` y `remove_item()` actualizan los índices `limSup` y `limInf` del buffer circular y modifican la variable `cantidad` sin ninguna protección, lo que ilustra de manera directa el riesgo de condiciones de carrera cuando varios hilos acceden simultáneamente a la misma región.

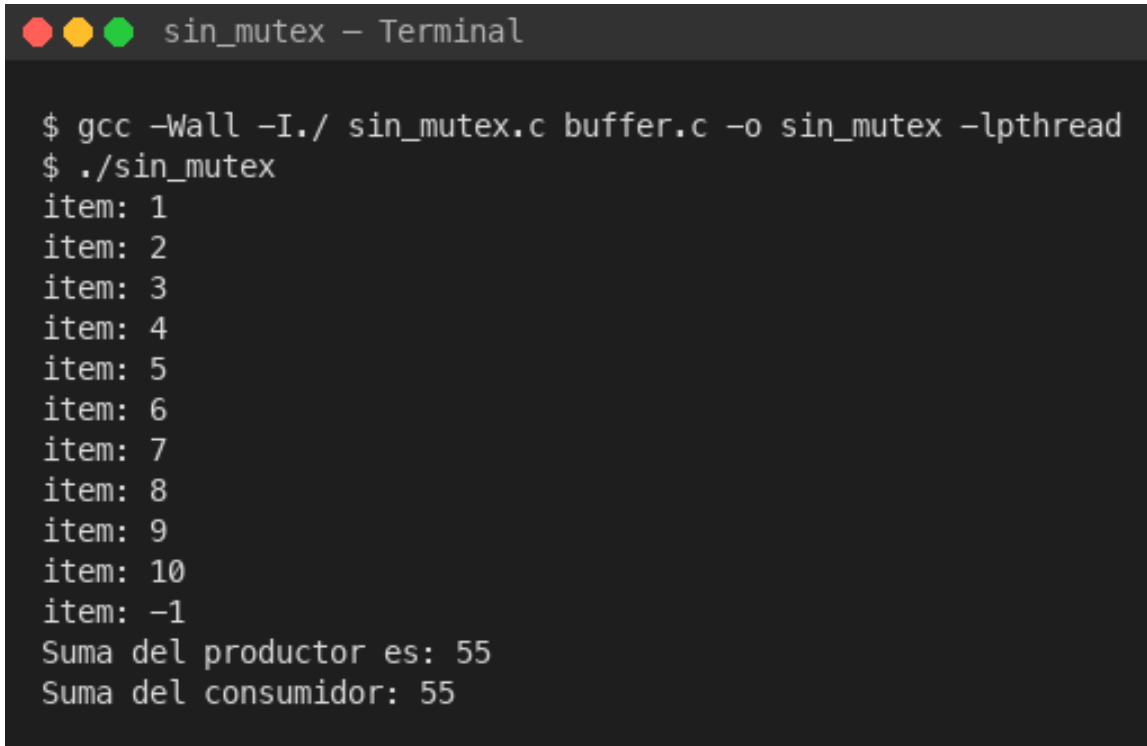
El archivo `sin_mutex.c` contiene la lógica esencial. La función `main` inicializa el buffer y las variables de los extremos de la cola, lanza los hilos productor y consumidor y espera a que terminen con `pthread_join`. El hilo productor ejecuta iterativamente la siguiente secuencia: produce un item leyendo del archivo, comprueba mediante un bucle `while` la condición `cantidad >= N` y, cuando hay hueco, inserta el elemento en el buffer. Para señalar el final se emplea la variable global `final`, de forma que el consumidor sepa que ya no llegarán más elementos.

El hilo consumidor realiza lo análogo: mientras el buffer esté vacío y el productor no haya terminado mantiene una espera consultando periódicamente la variable `cantidad`. Cuando encuentra elementos disponibles los extrae con `remove_item()` y los añade a su acumulador. El uso de `sleep` en la espera del consumidor reduce parcialmente el consumo de CPU, pero no elimina la naturaleza activa de la sincronización.

Al no existir exclusión mutua, es posible que productor y consumidor entren en la región crítica de manera simultánea. Operaciones como `cantidad++` o `cantidad--` no son atómicas y pueden provocar lecturas o escrituras entrelazadas que corrompan el estado del buffer. Se comprobó que, con ejecuciones de pocas iteraciones, el resultado puede coincidir con el esperado, pero al aumentar la presión sobre el buffer aparecen inconsistencias en las sumas finales, evidenciando el problema.

A modo de ejemplo, a continuación se muestra la salida obtenida al ejecutar la implementación con el archivo `numeros.txt` que contiene los enteros del 1 al 10, caso en

el que, por la baja presión sobre el buffer, ambos acumuladores coinciden con la suma esperada (55).



```
sin_mutex - Terminal

$ gcc -Wall -I./ sin_mutex.c buffer.c -o sin_mutex -lpthread
$ ./sin_mutex
item: 1
item: 2
item: 3
item: 4
item: 5
item: 6
item: 7
item: 8
item: 9
item: 10
item: -1
Suma del productor es: 55
Suma del consumidor: 55
```

Conclusiones

La implementación demuestra que es posible construir una primera versión del productor-consumidor sin recurrir a primitivas de sincronización, pero pone de manifiesto dos limitaciones importantes. En primer lugar, la espera activa desperdicia ciclos de CPU al consultar reiteradamente una condición que solo cambia en otro hilo, penalizando el rendimiento global del sistema. En segundo lugar, la ausencia de exclusión mutua permite que ambos hilos accedan a la región crítica a la vez, dando lugar a condiciones de carrera que pueden provocar resultados incorrectos o estados inconsistentes del buffer.

Estas carencias motivan directamente el siguiente ejercicio, en el que se introducen mutexes y variables de condición para garantizar la exclusión mutua y eliminar la espera activa. La comparación entre ambas implementaciones permite valorar de forma práctica el coste de no disponer de mecanismos de sincronización y justifica su presencia en los sistemas operativos reales.